Flow graph:

Displays the communication flow between devices, making packing exchange easier to understand

Capture 100 Packets

- Open the capture file
- Apply the TCP || UDP filter
- Inspect and view the flow graph
- Open the capture file
- Apply the coop filter
- Inspect and view the flow graph
- Apply the DNS filter
- Inspect and view the flow graph
- Apply the IP or ICMP filter
- Inspect and view the flow graph. and same captars
- Apply the dhcp file
- Inspect and view the flow graph and save the capture

RESULT:

Thus the feature of wireshark and implement of different filter was succesfully compiled

Scanned with OKEN Scanner

Exp no: 6

6/8/26

ERROR DETECTION & CORRECTION - HAMMING CODE

AIM:

To write a program to implement error detection and correction using Hamming code

ALGORITHM:

1. Start the Program
2. Read the no of 4 bit data blocks (n) forming the input data stream.
3. For each 4 bit data block D = d_1 & d_2, d_3, d_4. Perform Hamming encoding.
4. Simulate transmission by introducing a single bit error at a chosen code word
5. At the receiver, recompute the check bits from the received code word:

$C_1 = P_1$ XOR d_1 XOR d_2 XOR d_4

$C_2 = P_2$ XOR d_1 XOR d_3 XOR d_4

$C_4 = P_4$ XOR d_2 XOR d_3 XOR d_4

6. Compute Error Position = C_4 C_2 C_1
7. If position $\neq 0$, no error has occured.
8. Extract the original 4 data bits from the Corrected 7-bit code word
9. Repeat steps 3 to 8 for all n blocks in the data stream
10. Stop the Program

Scanned with OKEN Scanner

PROGRAM:

```
def encode(d):
    h = [0] * 7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]

    error_pos = c1*1 + c2*2 + c4*4
    if error_pos != 0:
        print()
        h[error_pos - 1] ^= 1
    else:
        print("No error detected.")
    return error_pos

def to_str(bits):
    return "".join(str(b) for b in bits)

def main():
    if __name__ == "__main__":
```

Scanned with OKEN Scanner

OUTPUT :

```
===HAMMING CODE - ERROR DETECTION AND CORRECTION===
Enter number of 4 bit data blocks in the stream: 2

--- Block 1 ---
Enter 4 data bits : 1 0 1 1
original Data bits : 1 0 1 1
Encoded Hamming code : 0 1 1 0 0 1 1
Received code : 0 1 0 0 0 1 1
Corrected code : 0 1 1 0 0 1 1
Recovered data bits : 1 0 1 1

--- Block 2 ---
Enter 4 data bits : 0 1 1 0
original Data Bits : 0 1 1 0
Encoded Hamming Code : 1 1 0 0 1 1 0
Enter bit position : 6
Received code : 1 1 0 0 1 0 0
Corrected code : 1 1 0 0 1 1 0
Recovered Data Bits : 0 1 1 0
```

RESULT:

The Program to implement error detection and correction using the Hamming Code concept was successfully written in python, executed and verified using a sample input data stream

Scanned with OKEN Scanner